

ONEDRAFT

CAD-AI — Aria Powered

Executable Repository Foundation & Core Application Runtime

Version 1.0

Status: Implementation Specification

Predecessor: OneDraft Database Schema & OpenAPI Contract v0.1

Architecture Layer: Executable Foundation

Database: PostgreSQL

API: FastAPI / REST / JSON

API Version: /api/v1

Domain Language: Python

Validation Models: Pydantic

Identifiers: UUID

AI Layer: Aria Powered

Runtime Model: API + Worker + PostgreSQL + Redis

Primary Objective: Convert the established OneDraft architecture into an executable repository foundation.

1. IMPLEMENTATION POSITION

The OneDraft architecture has now passed through:

Product Definition

System Architecture

Technology Architecture

Domain Model

Persistence Model

OpenAPI Contract

Versioning Model

AI Command Boundary

Regulatory Model

Drawing Model

Redline Model

Validation Model

Document Provenance

Acceptance Workflow

Application Stack

The system now enters the executable repository stage.

The purpose of this stage is not to redesign OneDraft.

The purpose is to create the first functioning software skeleton capable of executing the previously established architecture.

The repository therefore becomes the first physical implementation of the OneDraft computational model.

2. EXECUTABLE FOUNDATION PRINCIPLE

The initial implementation shall preserve the separation between:

```
API
↓
APPLICATION SERVICES
↓
DOMAIN MODEL
↓
REPOSITORIES
↓
UNIT OF WORK
↓
POSTGRESQL
```

Long-running computation shall remain separated:

```
API
↓
JOB DISPATCH
↓
WORKER
↓
DOMAIN SERVICE
↓
RESULT
```

The AI layer shall remain separated:

```
ARIA
↓
COMMAND INTERPRETATION
↓
```

STRUCTURED COMMAND



COMMAND VALIDATION



PROJECT MODEL SERVICE



TRANSACTION

No AI provider shall receive direct authority over the database.

3. INITIAL REPOSITORY

The executable repository shall initially contain:

OneDraft/

```
├── README.md
├── LICENSE
├── pyproject.toml
├── .env.example
├── docker-compose.yml
├── .gitignore
├── apps/
│   ├── web/
│   ├── api/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   └── routes/
│   │       ├── __init__.py
│   │       └── projects.py
│   └── worker/
│       ├── __init__.py
│       └── worker.py
├── packages/
│   ├── domain/
│   │   ├── __init__.py
│   │   ├── domain_types.py
│   │   ├── commands.py
│   │   └── project_model_service.py
│   ├── persistence/
│   │   ├── __init__.py
│   │   ├── database.py
│   │   ├── repositories.py
│   │   └── unit_of_work.py
│   ├── compliance/
│   ├── geometry/
│   ├── documentation/
│   ├── validation/
│   └── shared/
```

```
├── migrations/
│   └── 001_initial_schema.sql
├── docs/
│   └── openapi/
│       └── openapi.yaml
└── tests/
    ├── integration/
    ├── domain/
    ├── persistence/
    └── api/
```

This structure establishes the initial monorepository boundary.

4. APPLICATION BOUNDARIES

The initial runtime contains three primary applications:

```
apps/web
apps/api
apps/worker
```

The web application provides the user interface.

The API provides the machine interface.

The worker executes asynchronous operations.

The domain packages contain business logic.

The persistence package contains database infrastructure.

No application shall bypass the domain boundary for project mutations.

5. PYTHON PACKAGE CONFIGURATION

The repository shall use:

```
pyproject.toml
```

The initial Python runtime shall target a current supported Python 3 release.

The project shall use:

```
FastAPI
Pydantic
SQLAlchemy
Alembic-compatible migration conventions
PostgreSQL driver
Redis client
```

Pytest

Additional computational libraries shall be introduced only when required by their corresponding subsystem.

6. ENVIRONMENT CONFIGURATION

The development environment shall be represented by:

`.env.example`

Initial variables:

```
APP_ENV=development
APP_NAME=onedraft
API_HOST=0.0.0.0
API_PORT=8000
```

```
DATABASE_URL=postgresql://onedraft:onedraft@postgres:5432/onedraft
```

```
REDIS_URL=redis://redis:6379/0
```

```
LOG_LEVEL=INFO
```

```
ARIA_PROVIDER=
ARIA_MODEL=
ARIA_API_KEY=
```

Secrets shall never be committed to source control.

The `.env.example` file contains configuration names only.

7. DATABASE BOOTSTRAP

The PostgreSQL database is initialized through:

```
migrations/001_initial_schema.sql
```

The migration shall create the logical schemas:

```
identity
projects
model
geometry
documentation
compliance
validation
revisions
audit
billing
```

system

The migration shall then create the tables established in the Database Schema & OpenAPI Contract v0.1.

The migration shall be deterministic.

Running the migration against a clean database must produce the same schema every time.

8. DATABASE EXTENSIONS

The first migration may enable required PostgreSQL extensions.

The initial implementation should use UUID generation supported by the selected PostgreSQL environment.

Where spatial functionality is introduced, the geometry subsystem shall determine whether PostGIS becomes a required dependency.

The initial architecture shall not force computational geometry into PostgreSQL merely because PostgreSQL is the transactional system of record.

9. DATABASE CONNECTION

The persistence package shall expose a centralized database configuration.

File:

`packages/persistence/database.py`

Responsibilities:

```
load DATABASE_URL
create database engine
create session factory
provide transactional session
provide health-check connection
```

The database configuration shall not contain domain logic.

10. SESSION MANAGEMENT

Database sessions shall be created per application operation.

The API shall never maintain one global mutable transaction for the lifetime of the process.

The worker shall independently acquire database sessions.

Transactions shall be explicitly bounded.

Conceptually:

```
REQUEST
↓
SESSION
↓
TRANSACTION
↓
DOMAIN OPERATION
↓
COMMIT / ROLLBACK
↓
SESSION CLOSE
```

11. UNIT OF WORK

File:

`packages/persistence/unit_of_work.py`

The Unit of Work establishes the transaction boundary.

Conceptually:

```
with unit_of_work() as uow:
    project = uow.projects.create(...)
    uow.commit()
```

If an exception occurs:

ROLLBACK

The Unit of Work shall provide repositories required by the transaction.

Initial repositories:

```
projects
requirements
buildings
levels
spaces
elements
constraints
relationships
revisions
commands
events
```

12. REPOSITORY PRINCIPLE

File:

`packages/persistence/repositories.py`

Repositories provide persistence operations without exposing raw database implementation details to the domain layer.

Conceptually:

ProjectRepository
RequirementRepository
BuildingRepository
LevelRepository
SpaceRepository
ElementRepository
ConstraintRepository
RevisionRepository
CommandRepository
EventRepository

Repositories shall not contain AI reasoning.

Repositories shall not contain regulatory interpretation.

Repositories shall not generate drawings.

Repositories persist domain state.

13. DOMAIN TYPES

File:

`packages/domain/domain_types.py`

The domain package shall contain the canonical typed representation of core OneDraft objects.

Initial types:

Project
Requirement
Building
Level
Space
Element
Assembly
Constraint
Relationship
ModelVersion

Revision
ChangeRecord
AriaCommand
DesignProposal
Redline
ValidationResult
DocumentPackage
AcceptanceRecord

Each object shall use UUID identifiers.

Datetime fields shall use timezone-aware timestamps.

14. DOMAIN ENUMERATIONS

The domain layer shall establish controlled values for frequently used states.

Initial examples:

ProjectStatus
RevisionStatus
CommandStatus
ValidationStatus
RedlineStatus
DocumentStatus
AcceptanceStatus
JobStatus
ElementStatus

The application shall not rely on arbitrary strings when a value represents a controlled finite state.

15. COMMAND TYPES

File:

packages/domain/commands.py

The command layer represents operations that may change the project model.

Initial commands:

CreateProject
CreateRequirement
CreateBuilding
CreateLevel
CreateSpace
CreateElement
MoveElement
ModifyElement
DeleteElement
CreateConstraint

CreateRedline
ResolveRedline
GenerateDrawings
RunValidation
GenerateDocumentPackage

Each command contains:

command_id
project_id
requested_by
expected_revision
operation
target_objects
parameters
preserve_constraints

16. COMMAND IMMUTABILITY

Once a command has entered execution, its original request payload shall be preserved.

The command may transition:

PROPOSED
↓
VALIDATING
↓
APPROVED
↓
EXECUTING
↓
COMPLETED

or:

PROPOSED
↓
REJECTED

or:

EXECUTING
↓
FAILED

The original command must remain auditable.

17. PROJECT MODEL SERVICE

File:

packages/domain/project_model_service.py

This becomes the primary application boundary for mutations to the OneDraft computational project model.

The service shall be responsible for:

- loading current project state
- checking revision state
- checking command validity
- checking active constraints
- applying model mutations
- creating change records
- creating new model versions
- creating revisions
- recording events

It shall not directly expose SQL.

18. PROJECT CREATION

The first executable project operation shall support:

CreateProject

The operation shall create:

- Project
- Initial Model Version
- Initial Revision

The relationship becomes:

```
PROJECT
  ↓
MODEL VERSION 1
  ↓
REVISION 1
```

The project shall receive a UUID.

19. PROJECT MUTATION

Every project mutation must specify the expected current revision.

Example:

expected_revision = 14

The service compares:

requested revision

against:

current project revision

If they differ, the command shall not silently modify the newer state.

This establishes optimistic concurrency control.

20. MODEL VERSION CREATION

A successful model mutation creates a new model version.

Conceptually:

MODEL VERSION 14

↓

COMMAND

↓

MODEL VERSION 15

The previous model version remains immutable.

21. REVISION CREATION

A completed model mutation creates a revision record.

Conceptually:

Revision 14

↓

Move Element

↓

Revision 15

The revision contains:

parent_revision_id

model_version_id

created_by

reason

status

validation_summary

code_manifest_id

22. CHANGE RECORD

Every mutation shall generate a change record.

Example:

```
object_id
operation = MOVE_ELEMENT
before_state = {...}
after_state = {...}
reason = "User requested wall relocation"
source = "ARIA"
```

The change record becomes part of historical reconstruction.

23. EVENT RECORDING

Every significant domain operation shall create an audit event.

Examples:

```
PROJECT_CREATED
MODEL_VERSION_CREATED
REVISION_CREATED
ARIA_COMMAND_CREATED
ARIA_COMMAND_COMPLETED
ELEMENT_MODIFIED
VALIDATION_STARTED
VALIDATION_COMPLETED
DRAWING_GENERATION_STARTED
DOCUMENT_PACKAGE_GENERATED
CUSTOMER_ACCEPTED
```

The event record shall be append-only.

24. ARIA BOUNDARY

Aria shall not directly mutate:

```
PostgreSQL
Redis
filesystem
production configuration
permissions
billing
```

Aria produces structured intent.

The application validates that intent.

The Project Model Service executes the permitted operation.

The architecture therefore remains:

```
USER
↓
ARIA
↓
STRUCTURED COMMAND
↓
COMMAND VALIDATION
↓
PROJECT MODEL SERVICE
↓
UNIT OF WORK
↓
DATABASE
```

25. ARIA REQUEST FLOW

The API endpoint:

```
POST /api/v1/projects/{project_id}/aria/requests
```

receives natural-language intent.

Example:

```
Move the rear bedroom wall 300 mm north while keeping the exterior envelope
unchanged.
```

The AI orchestration layer transforms that intent into structured command data.

Example:

```
operation = MOVE_ELEMENT
distance = 300
unit = mm
direction = NORTH
preserve_constraints = [...]
```

The structured command is then passed to the domain boundary.

26. COMMAND VALIDATION

Before execution the system validates:

project exists
user authorized
revision current
target objects exist
target objects belong to project
parameters valid
constraints identified
command permitted

The command must fail atomically when required validation fails.

27. CONSTRAINT PRESERVATION

Commands may identify constraints that must remain satisfied.

Example:

```
preserve_constraints:  
    exterior_envelope  
    property_setback  
    structural_grid
```

The command engine must not treat these values as merely descriptive metadata.

They become execution constraints.

28. API APPLICATION

File:

apps/api/main.py

The API application shall initialize:

```
FastAPI  
configuration  
logging  
database connectivity  
route registration  
health endpoint  
exception handling
```

The API root shall expose:

/api/v1

29. PROJECT ROUTES

File:

`apps/api/routes/projects.py`

Initial endpoints:

```
POST    /api/v1/projects
GET     /api/v1/projects/{project_id}
GET     /api/v1/projects
POST    /api/v1/projects/{project_id}/requirements
POST    /api/v1/projects/{project_id}/aria/requests
POST    /api/v1/projects/{project_id}/validation
POST    /api/v1/projects/{project_id}/drawings/generate
POST    /api/v1/projects/{project_id}/documents/generate
```

The initial route module may be expanded as additional resources become executable.

30. API HEALTH

The API shall expose a health endpoint.

Conceptually:

```
GET /health
```

The endpoint reports application availability.

A deeper readiness endpoint may verify:

```
PostgreSQL
Redis
worker connectivity
```

without exposing credentials or internal infrastructure details.

31. API ERROR HANDLING

All domain errors shall be translated into the standard OneDraft error structure.

Example:

```
CONSTRAINT_CONFLICT
REVISION_CONFLICT
PROJECT_NOT_FOUND
OBJECT_NOT_FOUND
COMMAND_INVALID
AUTHORIZATION_FAILED
```


VALIDATION_FAILED
JOB_NOT_FOUND

The API shall not expose raw database exceptions to clients.

32. IDEMPOTENCY

Mutating endpoints shall support:

Idempotency-Key

The key shall be associated with:

request_hash
response_hash
response_payload
created_at

A retry using the same key and equivalent request shall not create a duplicate mutation.

33. WORKER APPLICATION

File:

apps/worker/worker.py

The worker is responsible for asynchronous operations.

Initial operations:

geometry computation
validation
drawing generation
document generation
future export operations

The worker must operate through application/domain services.

It must not become a second independent implementation of project mutation logic.

34. JOB MODEL

Long-running operations produce:

job_id

Initial states:

QUEUED
RUNNING
VALIDATING
COMPLETED
FAILED
CANCELLED

The worker updates job state as execution progresses.

35. JOB ISOLATION

Every job shall identify its source state.

Minimum identifiers:

project_id
revision_id
model_version_id
job_id

Where applicable:

code_manifest_id
view_id
drawing_id

A worker must never assume that the current project state is identical to the state that generated its job.

36. STALE JOB PROTECTION

Suppose:

Revision 17

generates a drawing job.

Before the worker finishes:

Revision 18

becomes current.

The Revision 17 job must retain its original provenance.

It must not silently overwrite Revision 18 output.

37. REDIS

Redis shall initially provide:

- job queue
- job state coordination
- short-lived cache
- distributed execution coordination

Redis shall not become the authoritative project database.

PostgreSQL remains the transactional system of record.

38. DOCKER COMPOSE

File:

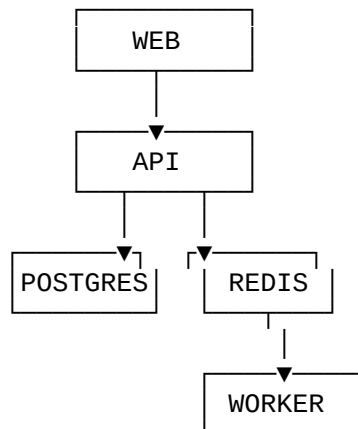
`docker-compose.yml`

The development stack shall initially provide:

- postgres
- redis
- api
- worker

The web application may be added as its executable implementation becomes available.

Conceptually:



39. LOCAL DEVELOPMENT

The complete development stack shall eventually start through:

```
docker compose up
```

The expected development sequence is:

```
START DATABASE
↓
RUN MIGRATIONS
↓
START REDIS
↓
START API
↓
START WORKER
↓
RUN INTEGRATION TESTS
```

40. FIRST DATABASE BOOTSTRAP TEST

The first automated persistence test shall verify:

```
database starts
schemas exist
tables exist
foreign keys exist
indexes exist
migration completes
```

A clean database must be able to reach the expected schema without manual intervention.

41. FIRST DOMAIN TEST

The first domain test shall execute:

```
Create Project
↓
Create Initial Model Version
↓
Create Initial Revision
```

and verify:

```
project.current_model_version_id
project.current_revision_id
```

correctly reference the newly created records.

42. FIRST MUTATION TEST

The next test shall execute:

```
Create Project
↓
Create Building
↓
Create Level
↓
Create Element
↓
Move Element
```

The test must verify:

```
old model version remains unchanged
new model version exists
new revision exists
change record exists
audit event exists
project current state advances
```

43. CONCURRENCY TEST

The system shall test:

```
Client A → Revision 14
Client B → Revision 14
```

Client A successfully creates:

```
Revision 15
```

Client B then attempts to mutate Revision 14.

The second command must receive:

```
REVISION_CONFLICT
```

The database must not contain a conflicting partial state.

44. ARIA COMMAND TEST

The first AI boundary test may bypass the external model provider and provide a structured command directly.

Example:

operation:
MOVE_ELEMENT

target:
wall UUID

distance:
300 mm

direction:
NORTH

The test verifies that Aria-originated commands enter the same Project Model Service used by non-AI commands.

This prevents creation of a separate AI mutation path.

45. API INTEGRATION TEST

The first API integration test shall execute:

```
POST /projects
↓
GET /projects/{id}
↓
POST /projects/{id}/aria/requests
↓
GET project
```

The final project state must correspond to the successful command.

46. OPENAPI IMPLEMENTATION

File:

docs/openapi/openapi.yaml

The OpenAPI specification shall describe:

authentication
projects
requirements
ARIA requests
commands
revisions
jobs
validation
drawings
documents
errors

The specification must correspond to the actual API implementation.

The OpenAPI file shall not become an aspirational document disconnected from executable routes.

47. GENERATED API TYPES

The API contract shall become the basis for generated client types.

The frontend shall eventually consume generated TypeScript models.

The Python backend shall use Pydantic models.

This establishes:

```
OPENAPI
↓
GENERATED TYPES
↓
CLIENT / SERVER CONTRACT
```

48. Pydantic BOUNDARY

Pydantic models shall validate:

```
HTTP requests
HTTP responses
command payloads
configuration
external service payloads
```

Pydantic validation shall occur before domain execution.

Domain invariants shall still be enforced by the domain layer.

Schema validation and domain validation are therefore distinct.

49. DOMAIN VALIDATION

Example:

A Pydantic model may verify:

```
distance is numeric
unit is valid
direction is recognized
```

The domain service must additionally verify:

target exists
target belongs to project
revision is current
movement is permitted
constraints can be preserved

Both layers are required.

50. DATABASE VALIDATION

Database constraints shall enforce fundamental persistence invariants.

Examples:

PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK

Application-level rules shall not replace database integrity where relational constraints can enforce the invariant directly.

51. PROVENANCE

Every generated artifact must remain traceable to its source state.

For example:

PDF
↓
Document Package
↓
Revision
↓
Model Version
↓
Project

Compliance provenance adds:

Code Manifest
↓
Code Sources
↓
Code Rules

Validation provenance adds:

Validation Run

52. DRAWING PROVENANCE

A drawing must identify:

project_id
view_id
revision_id
model_version_id

A sheet must identify its drawing associations.

The drawing engine must never treat a rendered image as the authoritative model.

53. DOCUMENT PROVENANCE

A document package shall identify:

project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
file_references
document_hash

The package therefore becomes a reproducible output of the computational state.

54. HASHING

Generated document packages shall receive a cryptographic hash.

The hash is used to establish:

artifact identity
integrity
reproducibility
historical comparison

The hash shall not replace the underlying revision or model identifiers.

55. FILE STORAGE

Large artifacts shall not be stored directly inside PostgreSQL rows.

The architecture shall use object storage for:

- PDF
- DWG
- DXF
- SVG
- rendered images
- source documents
- large geometry artifacts
- future BIM exports

The database stores:

- storage reference
- hash
- metadata
- provenance

56. OBJECT STORAGE BOUNDARY

The object-storage service shall be accessed through an application abstraction.

Conceptually:

- DocumentStorage
- GeometryStorage
- SourceArtifactStorage

The domain layer should not depend on a particular cloud provider.

57. COMPLIANCE ENGINE BOUNDARY

The compliance package shall eventually expose:

- retrieve code source
- normalize code source
- create code manifest
- evaluate applicable rules
- return structured requirements

The compliance engine shall not directly modify architectural geometry.

It returns constraints and validation results to the project model.

58. VALIDATION ENGINE BOUNDARY

The validation package shall evaluate:

- model
- requirements
- constraints
- code manifest
- documentation

It produces:

- ValidationRun
- ValidationResult

The validation engine does not silently mutate the project model.

A corrective action becomes a separate command.

59. GEOMETRY ENGINE BOUNDARY

The geometry package shall provide computational geometry operations.

Initial conceptual operations:

- create geometry
- transform geometry
- intersect geometry
- offset geometry
- calculate bounds
- validate topology
- calculate areas
- calculate volumes

Geometry remains subordinate to the project model.

The geometry engine does not become the system of record.

60. DOCUMENTATION ENGINE BOUNDARY

The documentation package consumes:

- Project Model
- Views
- Drawing definitions
- Annotations
- Dimensions
- Schedules
- Sheet Templates

and produces:

Drawing
Sheet
Document Package

The documentation engine must be deterministic for identical model inputs and rendering configuration.

61. APPLICATION SERVICE BOUNDARY

The eventual service hierarchy becomes:

ProjectService
ModelService
CommandService
RevisionService
ComplianceService
ValidationService
DrawingService
DocumentService
AcceptanceService

The first executable implementation shall prioritize:

ProjectService
ModelService
CommandService
RevisionService

before the advanced computational engines.

62. OBSERVABILITY

The initial runtime shall record:

request_id
job_id
project_id
revision_id
model_version_id
command_id

where applicable.

Logs must permit an operation to be traced across:

API
↓
SERVICE

↓
DATABASE
↓
WORKER
↓
OUTPUT

63. STRUCTURED LOGGING

Logs should use structured fields rather than relying exclusively on free-form text.

Example:

```
event = command.completed  
project_id = UUID  
revision_id = UUID  
command_id = UUID  
duration_ms = 184  
status = COMPLETED
```

Sensitive credentials and secrets shall never be logged.

64. REQUEST IDENTIFICATION

Every API request shall receive:

`request_id`

The request identifier shall be propagated into application logging.

Asynchronous jobs shall additionally receive:

`job_id`

This creates traceability across synchronous and asynchronous execution.

65. SECURITY BOUNDARY

Authentication occurs before project access.

Authorization occurs before project mutation.

Project membership is verified against:

```
organization  
project  
user
```

scope

The domain service must not rely solely on frontend controls.

66. AUTHORIZATION PRINCIPLE

The effective permission becomes:

```
USER
+
ORGANIZATION MEMBERSHIP
+
PROJECT ACCESS
+
API SCOPE
+
OPERATION PERMISSION
```

An AI command must be evaluated under the authorization context of the requesting user or service identity.

67. TESTING ARCHITECTURE

The repository shall contain:

```
tests/domain
tests/persistence
tests/api
tests/integration
```

Domain tests verify business invariants.

Persistence tests verify database behavior.

API tests verify HTTP contracts.

Integration tests verify complete workflows.

68. FIRST REGRESSION WORKFLOW

The first complete automated workflow shall be:

```
CREATE ORGANIZATION
↓
CREATE USER
↓
```

CREATE MEMBERSHIP
↓
CREATE PROJECT
↓
CREATE REQUIREMENTS
↓
CREATE BUILDING
↓
CREATE GROUND LEVEL
↓
CREATE LEVEL 1
↓
CREATE ROOMS
↓
CREATE WALLS
↓
CREATE DOORS
↓
CREATE WINDOWS
↓
CREATE MODEL VERSION
↓
CREATE REVISION
↓
SUBMIT ARIA COMMAND
↓
VALIDATE COMMAND
↓
CREATE NEW REVISION
↓
RUN VALIDATION
↓
GENERATE DRAWINGS
↓
CREATE REDLINE
↓
RESOLVE REDLINE
↓
REGENERATE DRAWINGS
↓
GENERATE DOCUMENT PACKAGE
↓
HASH PACKAGE
↓
RECORD ACCEPTANCE
↓
RECONSTRUCT PROJECT HISTORY

This is the first complete vertical integration path.

69. FAILURE REQUIREMENT

The integration test must intentionally exercise failures.

Examples:

invalid project
invalid object
invalid revision
stale revision
invalid command
constraint conflict
validation failure
worker failure
document generation failure
duplicate request

Every failure must leave the persistent model consistent.

70. ATOMICITY REQUIREMENT

No operation may leave a partial model mutation.

For example:

MOVE WALL

must not produce:

new wall position
+
missing revision

or:

new revision
+
missing change record

or:

database mutation
+
failed audit event

Required transactional records must succeed together.

71. ASYNCHRONOUS ATOMICITY

Long-running computation requires a different boundary.

The system may create:

JOB = QUEUED

inside a transaction.

The worker then executes the computational operation.

Finalization must verify that the result still corresponds to:

```
project_id  
revision_id  
model_version_id
```

before committing the derived artifact as current.

72. CURRENT STATE RULE

Only one model version may be designated as the project's current model version.

Only one revision may be designated as the project's current revision.

Historical versions remain available.

Derived outputs may reference historical versions.

73. RECONSTRUCTION REQUIREMENT

Given:

```
project_id  
revision_id  
model_version_id
```

the application must be able to reconstruct the associated project state.

The reconstruction process must not depend on the current project state.

This requirement protects historical reproducibility.

74. REPLAY PRINCIPLE

The system should eventually support command replay.

Conceptually:

```
INITIAL MODEL  
↓  
COMMAND 1  
↓  
COMMAND 2  
↓  
COMMAND 3
```

↓
COMMAND N
↓
TARGET MODEL VERSION

The event/change architecture provides the foundation for this capability.

Replay implementation may remain beyond the first executable milestone.

75. FIRST EXECUTABLE ARTIFACT PACKAGE

The initial repository implementation package shall consist of:

```
001_initial_schema.sql
openapi.yaml
domain_types.py
commands.py
project_model_service.py
database.py
repositories.py
unit_of_work.py
main.py
projects.py
worker.py
docker-compose.yml
.env.example
pyproject.toml
```

These files constitute the first executable OneDraft foundation.

76. IMPLEMENTATION ORDER

The artifacts shall be implemented in dependency order:

```
001_initial_schema.sql
↓
database.py
↓
repositories.py
↓
unit_of_work.py
↓
domain_types.py
↓
commands.py
↓
project_model_service.py
↓
projects.py
```

↓
main.py
↓
worker.py
↓
docker-compose.yml
↓
integration tests

The OpenAPI definition remains the machine-readable external contract throughout implementation.

77. FIRST BUILD TARGET

The first successful local build must achieve:

`docker compose up`

followed by:

database available
redis available
API available
worker available
health endpoint available
migration complete

No CAD computation is required for this first runtime milestone.

The objective is a functioning application skeleton.

78. FIRST FUNCTIONAL TARGET

The first functional milestone is:

`POST /api/v1/projects`

creating a real project in PostgreSQL.

The response must contain:

project_id
revision_id
model_version_id

The project must then be retrievable through:

`GET /api/v1/projects/{project_id}`

This proves that:

API
↓
DOMAIN
↓
UNIT OF WORK
↓
REPOSITORY
↓
POSTGRESQL

is operational.

79. SECOND FUNCTIONAL TARGET

The second milestone is the first model mutation.

The system must execute:

CREATE PROJECT
↓
CREATE BUILDING
↓
CREATE LEVEL
↓
CREATE ELEMENT
↓
MODIFY ELEMENT

and create:

Revision 1
Revision 2

with complete historical provenance.

80. THIRD FUNCTIONAL TARGET

The third milestone is the asynchronous boundary.

The system must execute:

REQUEST
↓
JOB CREATED
↓
REDIS
↓
WORKER
↓
PROCESS
↓

RESULT



JOB COMPLETED

The first worker operation may be a simple deterministic test operation.

The architecture must prove that asynchronous computation works before geometry, compliance, and document engines are attached.

81. FOURTH FUNCTIONAL TARGET

The fourth milestone is the Aria boundary.

The system must execute:

USER REQUEST



ARIA INTERPRETATION



STRUCTURED COMMAND



DOMAIN VALIDATION



PROJECT MODEL MUTATION



REVISION



AUDIT EVENT

The AI provider itself remains replaceable.

82. FIFTH FUNCTIONAL TARGET

The fifth milestone is the first derived artifact.

The system shall generate a basic documentation representation from the model.

The sequence becomes:

MODEL



VIEW



DRAWING



SHEET

The first rendering may be intentionally simple.

The important requirement is that the drawing is derived from the model.

83. SIXTH FUNCTIONAL TARGET

The sixth milestone introduces compliance.

The sequence becomes:

```
PROJECT
↓
JURISDICTION
↓
CODE MANIFEST
↓
APPLICABLE RULES
↓
VALIDATION
↓
RESULTS
```

The code manifest remains versioned and immutable once attached to a finalized revision.

84. SEVENTH FUNCTIONAL TARGET

The seventh milestone is the complete document chain:

```
PROJECT
↓
MODEL VERSION
↓
REVISION
↓
CODE MANIFEST
↓
VALIDATION
↓
DRAWINGS
↓
DOCUMENT PACKAGE
↓
HASH
↓
ACCEPTANCE
```

This establishes the first complete OneDraft production lineage.

85. DEFINITION OF DONE

The executable foundation is complete when:

PostgreSQL initializes
Redis initializes
API initializes
Worker initializes
OpenAPI loads
Project creation works
Project retrieval works
Model mutation works
Revisioning works
Change records work
Audit events work
Optimistic concurrency works
Jobs work
Aria commands enter the domain boundary
No AI component directly writes the database
Historical project state can be reconstructed

86. ARCHITECTURAL NON-NEGOTIABLES

OneDraft shall not become:

AI → SQL

It shall remain:

AI
↓
COMMAND
↓
DOMAIN
↓
TRANSACTION
↓
DATABASE

OneDraft shall not become:

DRAWING → DATABASE

It shall remain:

PROJECT MODEL
↓
DOCUMENTATION ENGINE
↓
DRAWING

OneDraft shall not become:

CURRENT STATE ONLY

It shall remain:

CURRENT STATE

+
HISTORICAL STATE
+
DERIVED STATE
+
EVENT STATE

87. STACK POSITION

The OneDraft stack now becomes:

USER / WEB
REST API
APPLICATION SERVICES
ARIA COMMAND BOUNDARY
DOMAIN PROJECT MODEL
COMPLIANCE / VALIDATION
GEOMETRY / DOCUMENTATION
REPOSITORY / UNIT OF WORK
POSTGRESQL
REDIS / ASYNC JOBS
OBJECT STORAGE

Aria is therefore an intelligence layer operating above the controlled OneDraft application boundary.

88. CURRENT ENGINEERING STATE

OneDraft now has the following established architecture:

Product definition

System architecture

Technology architecture

Domain model

Database schema

OpenAPI contract

Versioning architecture

AI command boundary

Regulatory architecture

Drawing architecture

Redline architecture

Validation architecture

Document provenance

Acceptance workflow

Application runtime architecture

Executable repository architecture

The system has now crossed from architectural specification into implementation.

89. NEXT IMPLEMENTATION ARTIFACTS

The immediate executable artifacts are:

migrations/001_initial_schema.sql

docs/openapi/openapi.yaml

packages/domain/domain_types.py

packages/domain/commands.py

packages/domain/project_model_service.py

packages/persistence/database.py

packages/persistence/repositories.py

packages/persistence/unit_of_work.py

apps/api/main.py

apps/api/routes/projects.py

apps/worker/worker.py

docker-compose.yml

.env.example

pyproject.toml

These are no longer conceptual architecture components.

They are the source files that instantiate the architecture.

90. FINAL ARCHITECTURAL DECISION

The next OneDraft development stage is therefore not another conceptual redesign.

It is:

```
ARCHITECTURE
↓
SOURCE TREE
↓
MIGRATION
↓
DOMAIN TYPES
↓
COMMAND BOUNDARY
↓
TRANSACTION SERVICE
↓
API
↓
WORKER
↓
LOCAL RUNTIME
↓
INTEGRATION TEST
```

The objective is to produce the first running OneDraft system in which a real project can be created, persisted, versioned, modified through the controlled command boundary, and reconstructed from its historical state.

That is the foundation upon which the geometry engine, regulatory engine, drawing engine, redline engine, document engine, and full Aria orchestration layer will subsequently execute.

91. SPECIFICATION STATUS

ONEDRAFT EXECUTABLE REPOSITORY FOUNDATION v0.1

STATUS: ESTABLISHED

The architectural stack has now advanced from specification into executable implementation.

The immediate implementation target is the construction of the repository artifacts defined above, followed by:

```
DATABASE BOOTSTRAP
↓
```

APPLICATION STARTUP
↓
PROJECT CREATION
↓
MODEL MUTATION
↓
REVISION CONTROL
↓
ASYNC JOB EXECUTION
↓
ARIA COMMAND EXECUTION
↓
FIRST DRAWING GENERATION
↓
FIRST VALIDATION RUN
↓
FIRST DOCUMENT PACKAGE
↓
END-TO-END REGRESSION TEST